2 Exploring the microservices world with Spring Cloud

This chapter covers

- Learning about Spring Cloud's technologies
- Understanding the principles of cloud-native applications
- Applying the twelve-factor app best practices
- Using Spring Cloud to build microservices

Designing, implementing, and maintaining microservices quickly become a problem if these processes are not managed correctly. When we start working with microservice solutions, it is essential to apply *best practices* to keep the architecture as efficient and scalable as possible to avoid performance issues, bottlenecks, or operational problems. Adhering to best practices also makes it easier for new developers to come up to speed with our systems. As we continue our discussion of microservices architectures, it's important to keep the following in mind:

The more distributed a system is, the more places it can fail.

By this we mean that with a microservice architecture, we have more points of failure. That's because instead of having a single monolith application, we now have an ecosystem of multiple individual services that interact with each other. This is the main reason why developers often encounter different administration and synchronization challenges or points of failure when creating microservice applications or architectures. To avoid possible points of failure, we will use Spring Cloud. Spring Cloud offers a set of features (service registration and discovery, circuit breakers, monitoring, and others) that will allow us to quickly build microservice architectures with minimal configurations.

This chapter briefly introduces the Spring Cloud technologies that we will use throughout this book. This is a high-level overview; as you use the different technologies, we'll give you the details on each as needed. Because we will use microservices throughout the next chapters, it is crucial to understand the concept of microservices, their benefits, and their development patterns.

2.1 What is Spring Cloud?

Implementing all of the patterns we explained in the first chapter from scratch would be a tremendous amount of work. Fortunately, for us, the Spring team has integrated a wide number of battle-tested open source projects into a single Spring subproject collectively known as Spring Cloud (https://projects.spring.io/spring-cloud/).

Spring Cloud is a collection of tools that wraps the work of open source companies such as VMware, HashiCorp, and Netflix in delivery patterns. Spring Cloud simplifies setting up and configuring our projects and provides solutions to the most commonly encountered patterns into our Spring application. We can then focus on writing code, not on getting buried in the details of configuring all the infrastructure that goes into building and deploying a microservice application. Figure 2.1 maps the patterns listed in the previous chapter to the Spring Cloud projects that implement them.

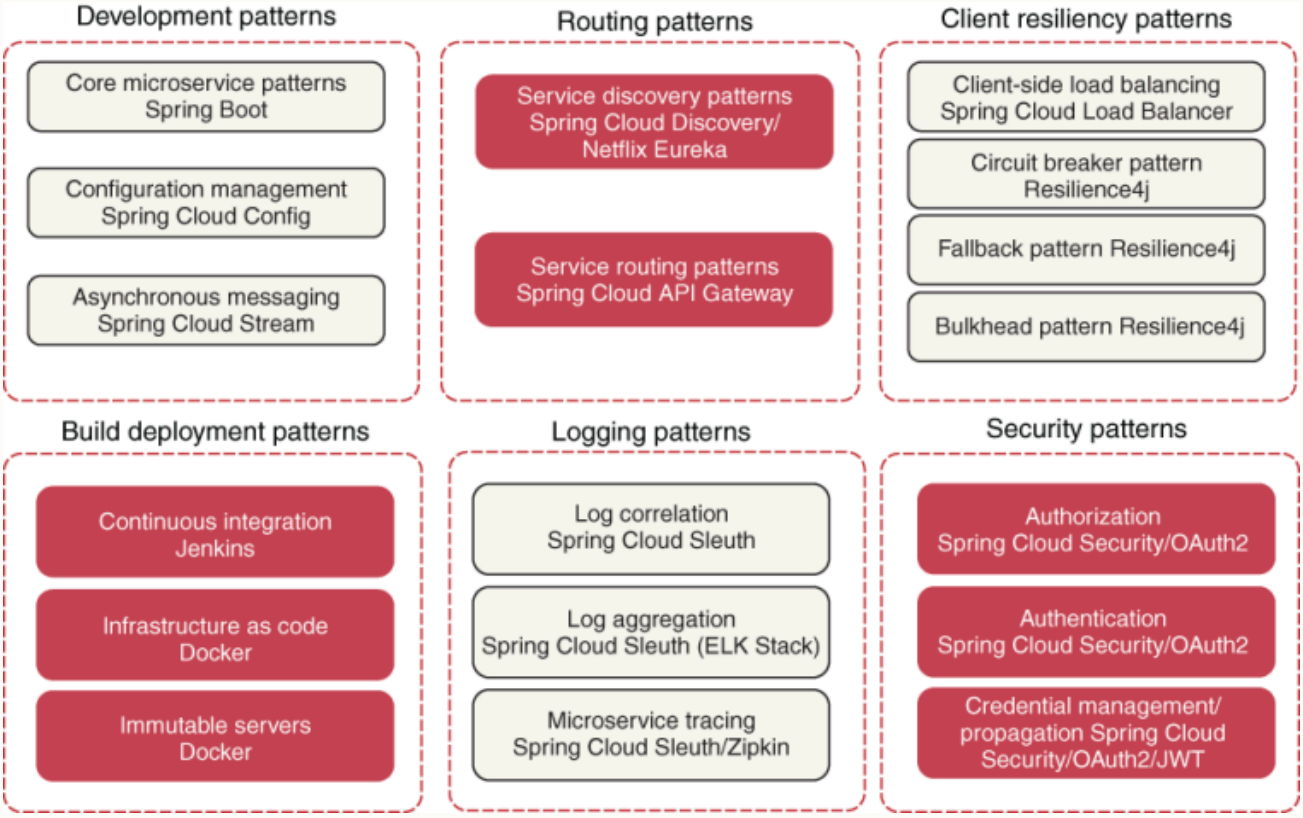


Figure 2.1 With Spring Cloud, we can map the technologies we're going to use directly to the microservice patterns we've explored so far.

2.1.1 Spring Cloud Config

Spring Cloud Config handles the management of the application configuration data through a centralized service. Your application configuration data (particularly your environment-specific configuration data) is then cleanly separated from your deployed microservice. This ensures that no matter how many microservice instances you bring up,

they'll always have the same configuration. Spring Cloud Config has its own property management repository but also integrates with open source projects like these:

- *Git (**https://git-scm.com/**)*—An open source version control system that lets you manage and track changes to any text file. Spring Cloud Config integrates with a Git backend repository and reads the application's configuration data from the repository.
- *Consul (**https://www.consul.io/**)*—An open source service discovery that allows service instances to register themselves with a service. Service clients can then query Consul to find the location of their service instances. Consul also includes a key-value store database that Spring Cloud Config uses to store application configuration data.
- *Eureka (**https://github.com/Netflix/eureka**)*—An open source Netflix project that, like Consul, offers similar service discovery capabilities. Eureka also has a key-value database that can be used with Spring Cloud Config.

1.2 Spring Cloud Service Discovery

With Spring Cloud Service Discovery, you can abstract away the physical location (IP and/or server name) of where your servers are deployed from the clients consuming the service. Service consumers invoke business logic for the servers through a logical name rather than a physical location. Spring Cloud Service Discovery also handles the registration and deregistration of service instances as these are started and shut down. Spring Cloud Service Discovery can be implemented using the following services:

- Consul (**https://www.consul.io/**)
- Zookeeper (**https://spring.io/projects/spring-cloud-zookeeper**)
- Eureka (**https://github.com/Netflix/eureka**) as its service discovery engine

NOTE Although Consul and Zookeeper are powerful and flexible, the Java developer community still uses Eureka. This book contains examples with Eureka to keep it manageable and material focused, and to reach the largest audience possible. If you're interested in Consul or Zookeeper, make sure you read appendixes C and D. In appendix C, we added an example of how we can use Consul as a service discovery, and in appendix D, we added an example of how to use Zookeeper.

1.3 Spring Cloud LoadBalancer and Resilience4j

Spring Cloud integrates heavily with several open source projects. For microservice client resiliency patterns, Spring Cloud wraps the Resilience4j library and the Spring Cloud LoadBalancer project to make using these within your own microservices trivial to implement. You'll find the Resilience4j library here:

https://github.com/resilience4j/resilience4j. By using the Resilience4j libraries, you can quickly implement service client resiliency patterns such as circuit breaker, retry, bulkhead, and more.

While the Spring Cloud LoadBalancer project simplifies integrating with service discovery agents such as Eureka, it also provides client-side load balancing of calls from a service consumer. This makes it possible for a client to continue making service calls even if the service discovery agent is temporarily unavailable.

1.4 Spring Cloud API Gateway

The API Gateway provides service-routing capabilities for your microservice application. Like the name says, it is a service gateway that proxies service requests and makes sure that all calls to your microservices go through a single "front door" before the targeted service is invoked. With this centralization of service calls, you can enforce standard service policies such as security authorization, authentication, content filtering, and routing rules. You can implement the API Gateway using Spring Cloud Gateway (https://spring.io/projects/spring-cloud-gateway).

NOTE In this book, we use the Spring Cloud API Gateway that was built with Spring Framework 5 Project Reactor (allowing integration with Spring Web Flux) and Spring Boot 2 to better integrate our Spring projects.

1.5 Spring Cloud Stream

Spring Cloud Stream (https://cloud.spring.io/spring-cloud-stream) is an enabling technology that lets you easily integrate lightweight message processing into your microservice. Using Spring Cloud Stream, you can build intelligent microservices that use asynchronous events as these occur in your application. You can also quickly integrate your microservices with message brokers such as RabbitMQ (https://www.rabbitmq.com) and Kafka (http://kafka.apache.org).

1.6 Spring Cloud Sleuth

Spring Cloud Sleuth (https://cloud.spring.io/spring-cloud-sleuth/) lets you integrate unique tracking identifiers into the HTTP calls and message channels (RabbitMQ, Apache Kafka) used within your application. These tracking numbers, sometimes referred to as correlation or trace IDs, allow you to track a transaction as it flows across the different services in your application. With Spring Cloud Sleuth, trace IDs are automatically added to any logging statements you make in your microservice.

The real beauty of Spring Cloud Sleuth is seen when it's combined with logging-aggregation technology tools like the ELK Stack (**https://www.elastic.co/what-is/elk-stack**) and tracking tools like Zipkin (**http://zipkin.io**). Open Zipkin takes data produced by Spring Cloud Sleuth and allows you to visualize the flow of your service calls involved for a single transaction. The ELK Stack is the acronym for three open source projects:

- Elasticsearch (**https://www.elastic.co**) is a search and analytics engine.
- Logstash (**https://www.elastic.co/products/logstash**) is a server-side, data-processing pipeline that consumes data and then transforms it in order to send it to a "stash."
- Kibana (**https://www.elastic.co/products/kibana**) is a client UI that allows the user to query and visualize the data of the whole stack.

1.7 Spring Cloud Security

Spring Cloud Security (**https://cloud.spring.io/spring-cloud-security/**) is an authentication and authorization framework that controls who can access your services and what they can do with them. Because Spring Cloud Security is token-based, it allows services to communicate with one another through a token issued by an authentication server. Each service receiving an HTTP call can check the provided token to validate the user's identity and their access rights. Spring Cloud Security also supports JSON Web Tokens (JWT). JWT (**https://jwt.io**) standardizes the format for creating an OAuth2 token and normalizes digital signatures for a generated token.

2.2 Spring Cloud by example

In the last section, we explained all the different Spring Cloud technologies that you're going to use to build your microservices. Because each of these technologies is an independent service, it will take more than one chapter to explain all of the details. However, as we wrap up this chapter, we want to leave you with a small code example that again demonstrates how easy it is to integrate these technologies into your own microservice developments.

Unlike the first code example in listing 1.1, you can't run this example because we first need to set up and configure a number of supporting services. Don't worry though; the setup costs for these Spring Cloud services are a one-time expense. Once they're set up, your individual microservices can use these capabilities over and over again. We couldn't fit all that goodness into a single code example at the beginning of the book. The following listing quickly demonstrates how we can integrate service discovery and client-side load balancing for remote services in our Hello World example.

Listing 2.1 Hello World service using Spring Cloud

```
package com.optima.growth.simpleservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.EnableEurekaClient;
import org.springframework.http.HttpMethod;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.client.RestTemplate;

@SpringBootApplication
@RestController
@RequestMapping(value="hello")
@EnableEurekaClient                                          (1)
public class Application {

  public static void main(String[] args) {
    SpringApplication.run(ContactServerAppApplication.class, args);
  }

  public String helloRemoteServiceCall(String firstName,String lastName){
    RestTemplate restTemplate = new RestTemplate();
    ResponseEntity<String> restExchange =                    (2)
      restTemplate.exchange(
        "http://logical-service-id/name/" + "{firstName}/
        {lastName}", HttpMethod.GET, null, String.class,
    firstName, lastName);
  return restExchange.getBody();
}

  @RequestMapping(value="/{firstName}/{lastName}",
                  method = RequestMethod.GET)
  public String hello(@PathVariable("firstName") String firstName,
                   @PathVariable("lastName") String lastName) {
    return helloRemoteServiceCall(firstName, lastName);
  }
}
```

❶ Tells the service to register with a Eureka service discovery agent to look up the location of remote services

❷ Uses a decorated RestTemplate class to take a "logical" service ID, and Eureka under the covers will look up the service's physical location

This code has a lot packed into it, so let's walk through it. Keep in mind that this listing is only an example and isn't found in the chapter 2 GitHub repository source code. We've included it here to give you a taste of what's to come later in the book.

The first thing to notice is the `@EnableEurekaClient` annotation. This annotation tells your microservice to register itself with a Eureka service discovery agent because you're going to use service discovery to look up remote REST service endpoints. Note that the configuration happens in a property file, giving the simple service the location and port number of a Eureka server to contact.

The second thing to notice is what's occurring inside the `helloRemoteServiceCall` method. The `@EnableEurekaClient` annotation tells Spring Boot that you are enabling the Eureka client. It is important to highlight that this annotation is optional if you already have the `spring-cloud-starter-netflix-eureka-client` dependency in your pom.xml. The `RestTemplate` class lets you pass in a logical service ID for the service you're trying to invoke, for example:

```
ResponseEntity<String> restExchange = restTemplate.exchange
    (http://logical-service-id/name/{firstName}/{lastName}
```

Under the covers, the `RestTemplate` class contacts the Eureka service and looks up the physical location of one or more of the named service instances. As a consumer of the service, your code doesn't need to know where that service is located.

The `RestTemplate` class also uses the Spring Cloud LoadBalancer library. This library retrieves a list of all the physical endpoints associated with a service. Every time the service is called by the client, it "round-robins" the call to different service instances without having to go through a centralized load balancer. By eliminating a centralized load balancer and moving it to the client, you eliminate another failure point (the load balancer going down) in your application infrastructure.

We hope that you're impressed now: you've added a significant number of capabilities to your microservice with only a few annotations. That's the real beauty behind Spring Cloud. You, as a developer, get to take advantage of battle-hardened microservice capa-

bilities from premier cloud companies like Netflix and Consul. Spring Cloud simplifies their use to literally nothing more than a few simple annotations and configuration entries. Before we start building our first microservice, let's look at the best practices for implementing a cloud-native microservice.

2.3 How to build a cloud-native microservice

In this section, we pause to understand the best practices for designing cloud microservice applications. In the previous chapter, we explained the difference between the cloud computing models, but what exactly is the cloud? The cloud is not a place; instead, it is a technology resource management system that lets you replace local machines and private data centers by using a virtual infrastructure. There are several levels or types of cloud applications, but in this section, we only focus on two types of cloud applications—cloud-ready and cloud-native.

A *cloud-ready* application is an application that was once used on a computer or on an onsite server. With the arrival of the cloud, these types of applications have moved from static to dynamic environments with the aim of running in the cloud. For example, a cloud-unready application could be a local on-premises application that only contains one specific database configuration, which must be customized in each installation environment (development, stage, production). In order to make an application like this cloud-ready, we need to externalize the application's configuration so that it quickly adapts to different environments. By doing this, we can ensure the application will run on multiple environments without changing any source code during the builds.

A *cloud-native* application (figure 2.2) is designed specifically for a cloud computing architecture to take advantage of all of its benefits and services. When creating this type of application, developers divide the functions into microservices with scalable components like containers, enabling these to run on several servers. These services are then managed by virtual infrastructures through the DevOps processes with continuous delivery workflows.

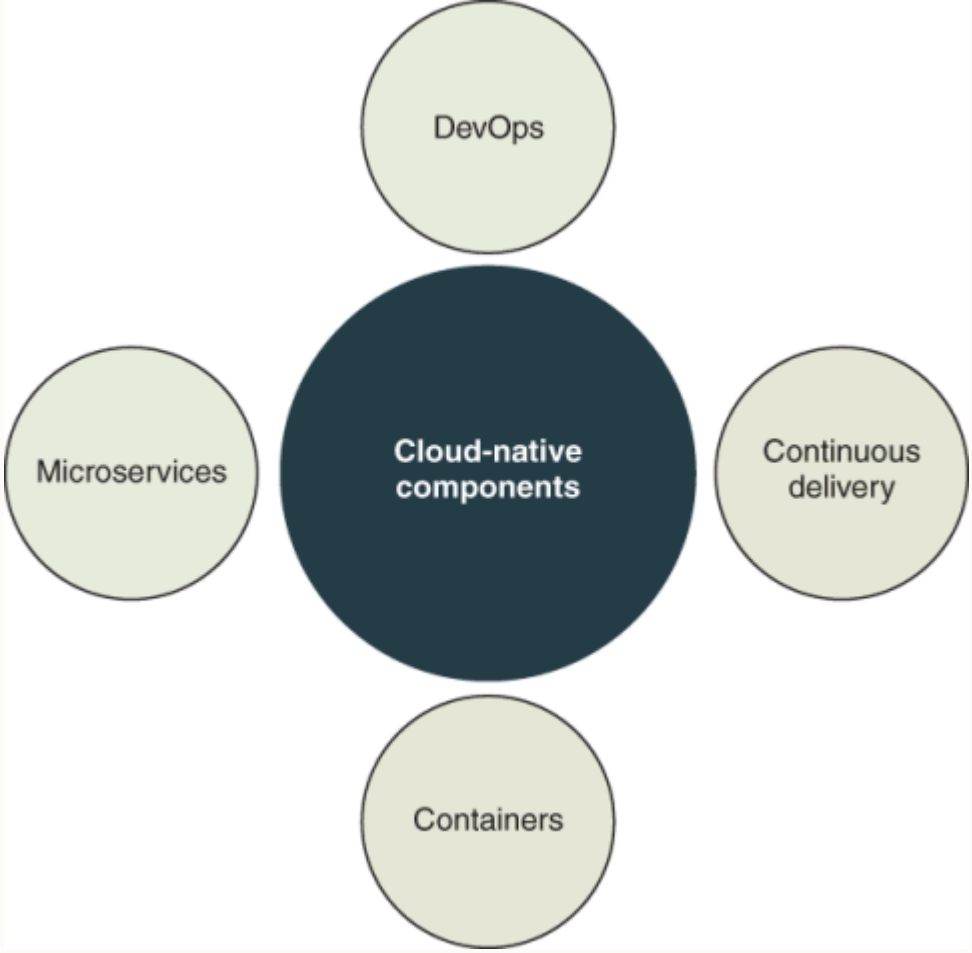


Figure 2.2 Cloud-native applications are built with scalable components like containers, deployed as microservices, and managed on virtual infrastructures through DevOps processes with continuous delivery workflows.

It's important to understand that cloud-ready applications do *not* require any change or conversion to work in the cloud. They are designed to deal with the unavailability of downstream components. The four principles of native cloud development are

- *DevOps is the acronym for development (Dev) and operations (Ops).* It refers to a software development methodology that focuses on communication, collaboration, and integration among software developers and IT operations. The main goal is to automate the software delivery processes and infrastructure changes at lower costs.
- *Microservices are small, loosely coupled, distributed services.* These allow you to take a large application and decompose it into easy-to-manage components with narrowly defined responsibilities. They also help combat the traditional problems of complexity in a large codebase by decomposing it down into small, well-defined pieces.
- *Continuous delivery is a software development practice.* With this practice, the process of delivering software is automated to allow short-term deliveries to a production environment.
- *Containers are a natural extension of deploying your microservices on a virtual machine (VM) image.* Rather than deploying a service to a full VM, many developers deploy their services as Docker containers (or similar container technology) to the cloud.

In this book, because we'll focus on creating microservices, we should remember that, by definition, these are cloud-native. This means that a microservice application can be executed on multiple cloud providers while obtaining all of the benefits of the cloud services.

In order to face the challenges of creating cloud-native microservices, we will use Heroku's best practice guide—called *the twelve-factor app*—in order to build high-quality microservices. The twelve-factor app (**https://12factor.net/**) enables us to build and develop cloud-native applications (microservices). We can see this methodology as a collection of development and designing practices that focus on dynamic scaling and fundamental points while building a distributed service.

This methodology was created in 2002 by several Heroku developers. The main goal was to provide 12 best practices when building microservices. We chose the twelve-factor document because it is one of the most complete guides to follow when creating cloud-native applications. This guide not only provides a common vocabulary about the most common problems observed in the development of modern applications, it also offers robust solutions to tackle these problems. Figure 2.3 shows the best practices covered by the twelve-factor manifesto.

NOTE In this chapter, we will give you a high-level overview of each best practice because, as you continue reading, you'll see how we intend to use the twelve-factor methodology throughout. Further, we'll apply those practices to examples with the Spring Cloud projects and other technologies.

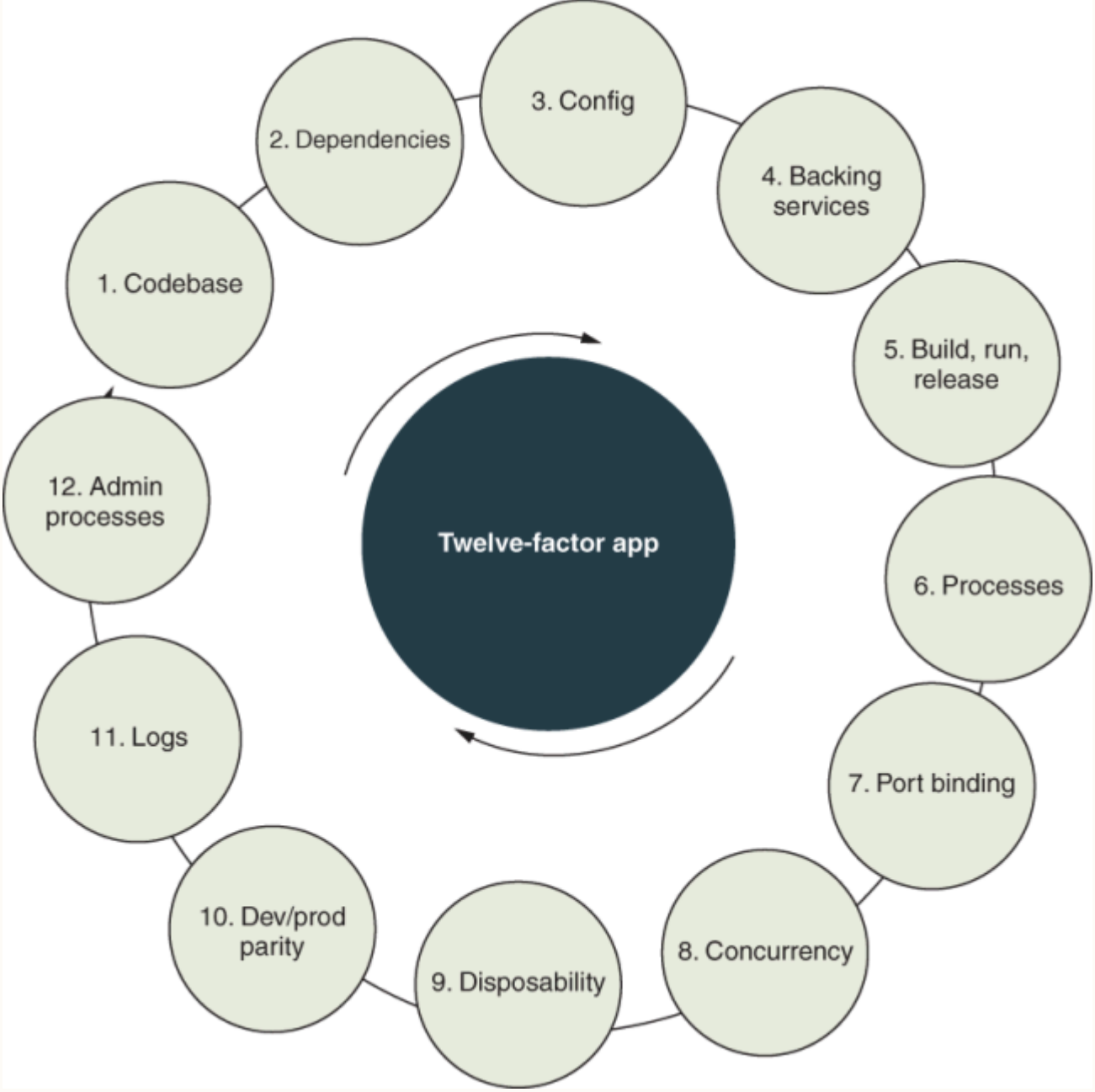


Figure 2.3 Twelve-factor application best practices

2.3.1 Codebase

With this practice, each microservice should have a single, source-controlled codebase. Also, it is essential to highlight that the server provisioning information should be in version control as well. Remember, *version control* is the management of changes to a file or set of files.

The codebase can have multiple instances of deployment environments (such as development, testing, staging, production, and so forth), but it's not shared with any other microservice. This is an important guideline because if we share the codebase for all the microservices, we would end up producing a lot of immutable releases that belong to different environments. Figure 2.4 shows a single codebase with many deployments.

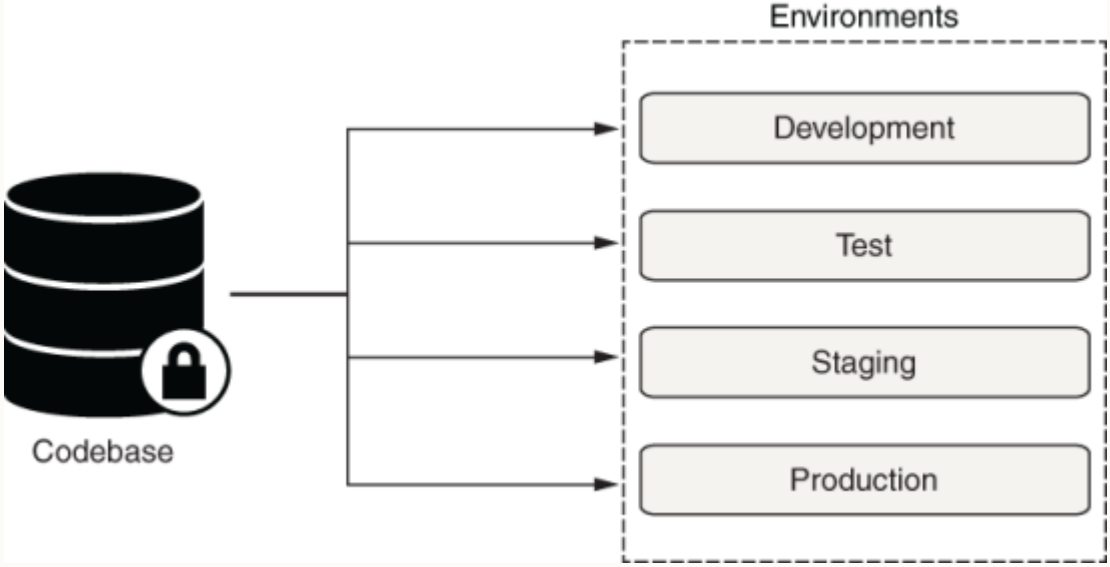


Figure 2.4 A single codebase with many deployments

3.2 Dependencies

This best practice explicitly declares the dependencies your application uses through build tools like Maven or Gradle (Java). Third-party JAR dependencies should be declared using their specific version numbers. This allows you to always build your microservices with the same library version.

If you are new to the build tools concept, figure 2.5 can help you understand how a build tool works. First, Maven reads the dependencies stored in the pom.xml file, then it searches for those on the local repository. If these are not found, then it proceeds to download the dependencies from the Maven central repository, inserting those into your local repository for future use.

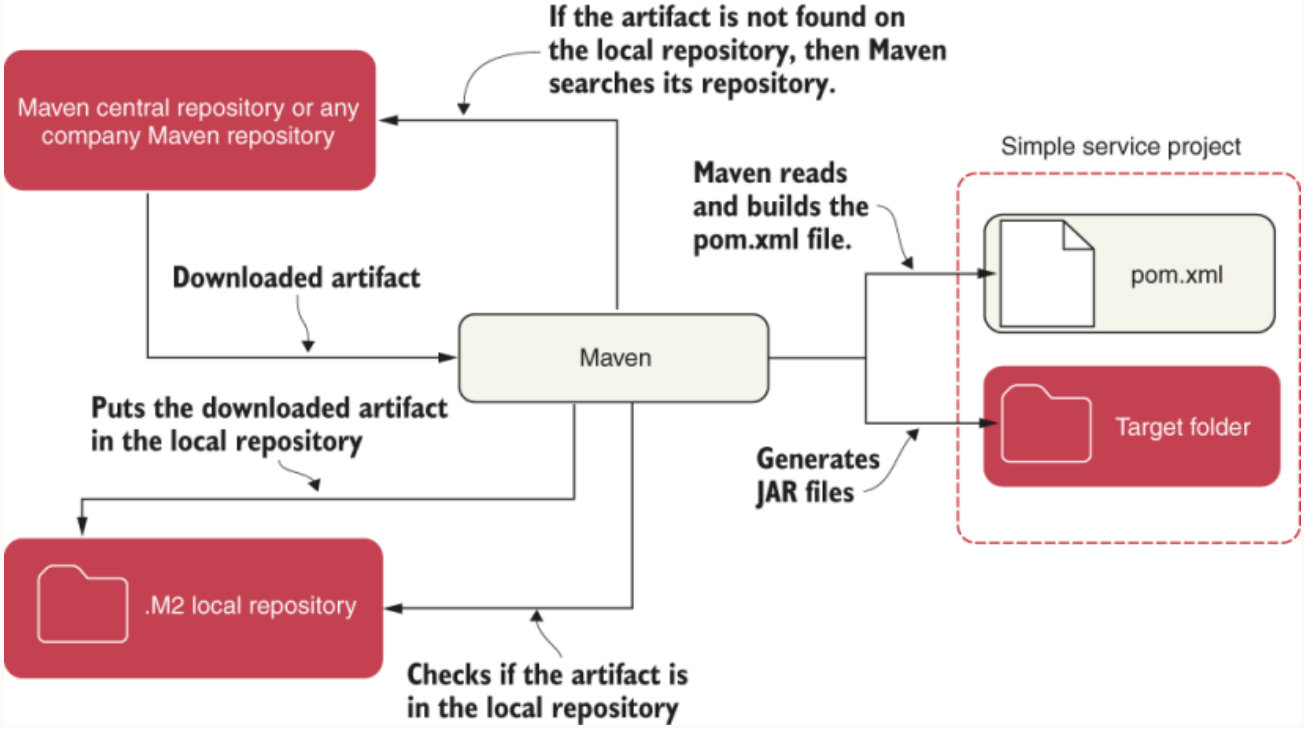


Figure 2.5 Maven reads the dependencies stored in the pom.xml file and then searches for them in a local repository. If the dependencies are not found, then Maven downloads the dependencies from the Maven repository, inserting them into your local repository.

3.3 Config

This practice refers to how you store your application configurations (especially your environment-specific configurations). Never add embedded configurations to your source code! Instead, it's best to maintain your configuration completely separate from your deployable microservice.

Imagine this scenario: you want to update a configuration for a specific microservice that has been replicated 100 times on a server. If you keep the configuration packaged within the microservice, you'll need to redeploy each of the 100 instances to make the change. *However, microservices can load the external configuration and use a cloud service to reload that configuration at run time without having to restart the microservice.* Figure 2.6 shows an example of how your environment should look.

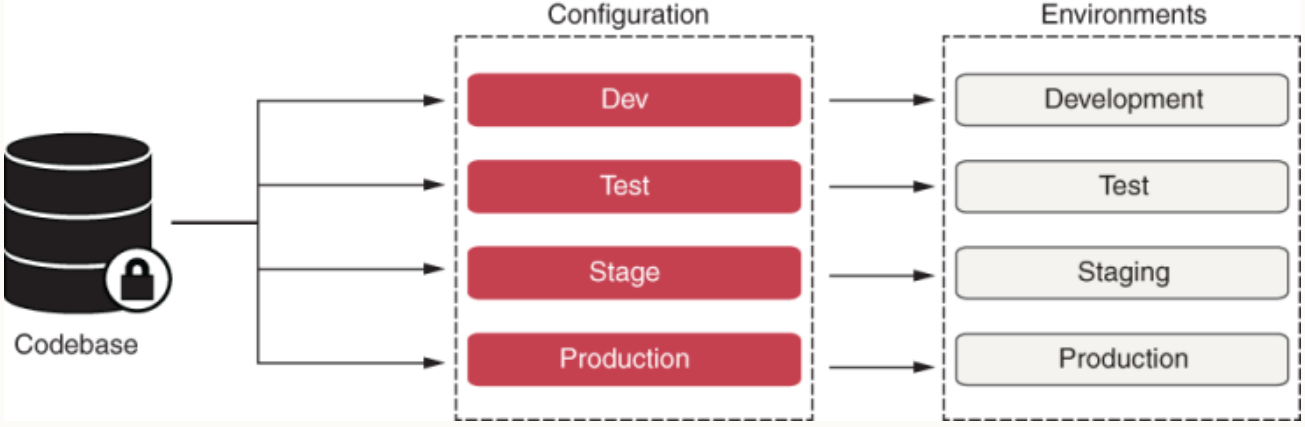


Figure 2.6 Externalizing environment-specific configurations

3.4 Backing services

Your microservice will often communicate over a network with databases, API RESTful services, other servers, or messaging systems. When it does, you should ensure that you can swap your deployment implementations between local and third-party connections without any changes to the application code. In chapter 12, we'll see how to move the microservices from a locally managed database to one managed by Amazon. Figure 2.7 shows an example of some backing services that our applications might have.

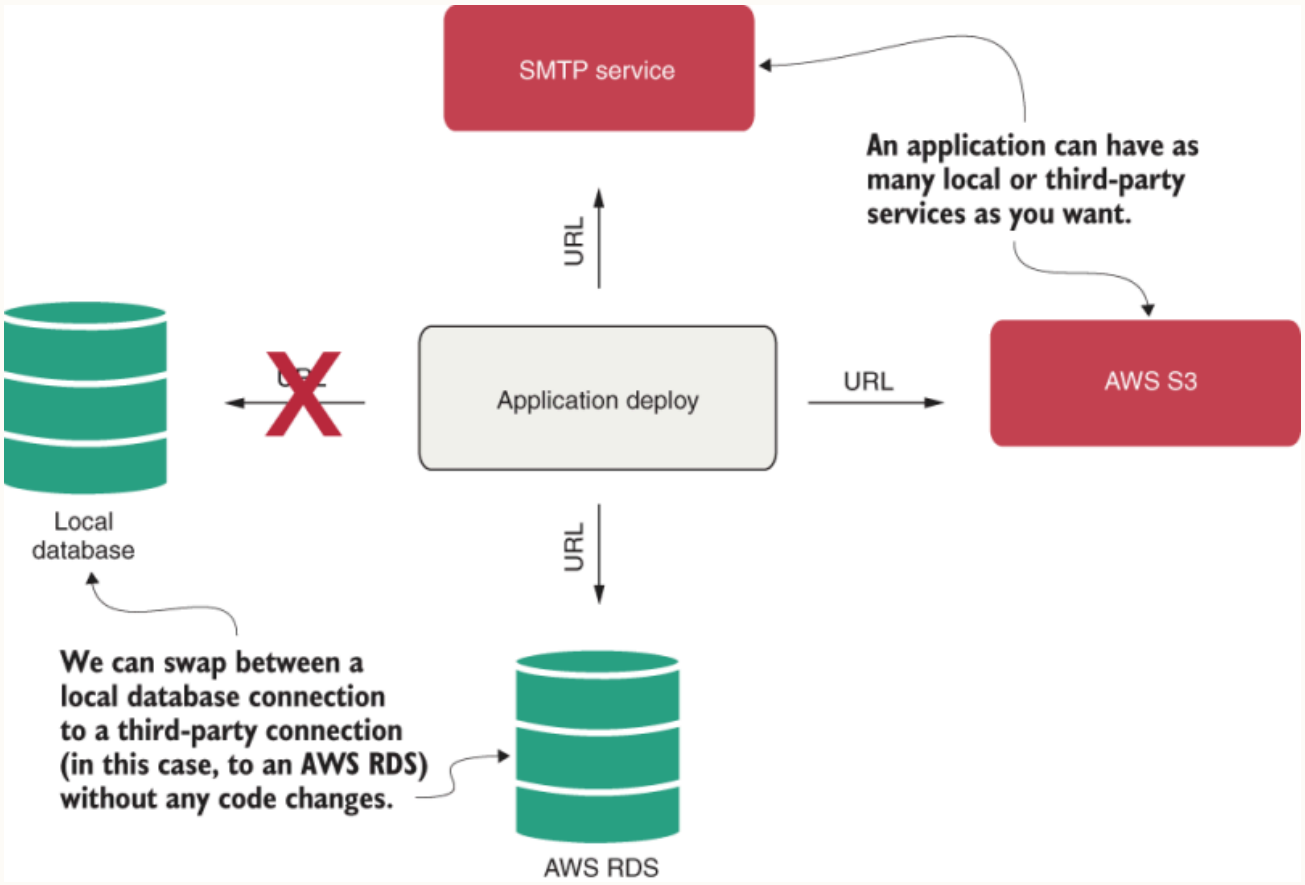


Figure 2.7 A backing service is any service that the application consumes over the network. When deploying the application, you should be able to swap a local connection to a third-party without any changes to the code.

3.5 Build, release, run

This best practice reminds us to keep our build, release, and run stages of application deployment completely separated. We should be able to build microservices that are independent of the environment on which they run. Once our code is built, any run-time changes need to go back to the build process and be redeployed. A built service is immutable and cannot be changed.

The release phase is in charge of combining the built service with a specific configuration for each targeted environment. When we do not separate the different stages, this can lead to problems and differences in the code that are not traceable or, at best, are hard to trace. For example, if we modify a service already deployed in production, the change will not be logged in the repository, and two situations can occur: changes get lost with newer versions of the service or we are forced to copy the changes to the new version of the service. Figure 2.8 shows a high-level architectural example for this best practice.

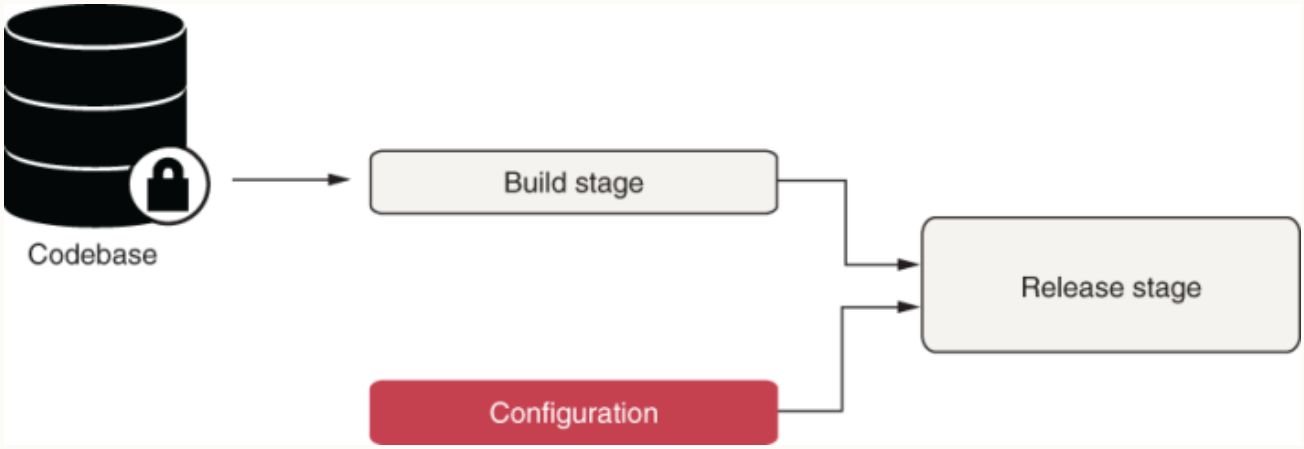


Figure 2.8 It's in our best practice to strictly separate the build, release, and run stages of the microservice.

3.6 Processes

Your microservices should always be stateless and should only contain the necessary information to carry out the requested transaction. Microservices can be killed and replaced at any time without the fear that a loss of a service instance will result in data loss. If there is a specific requirement to store a state, it must be done through an in-memory cache such as Redis or a backing database. Figure 2.9 shows how stateless microservices work.

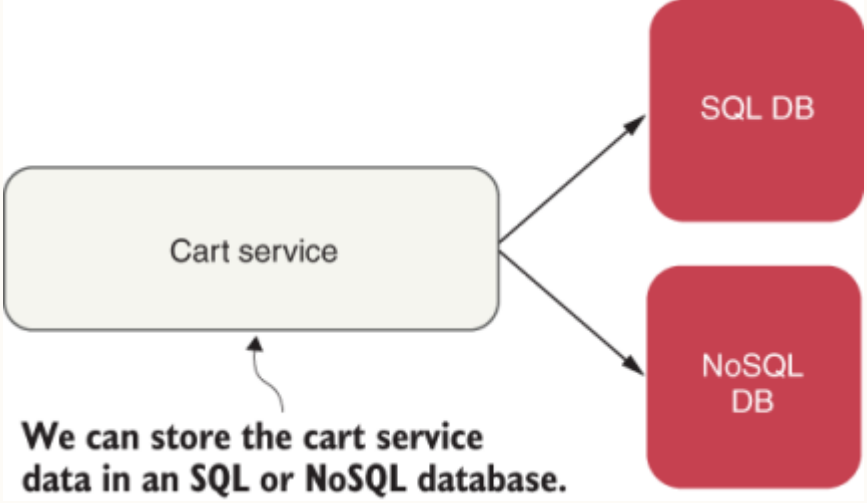


Figure 2.9 Stateless microservices don't store any session data (state) on the server. These services use SQL or NoSQL databases to store all the information.

3.7 Port binding

Port binding means to publish services through a specific port. In a microservices architecture, a microservice is completely self-contained with the run-time engine for the service packaged in the service executable. You should run the service without the need for a separate web or application server. The service should start by itself on the command line and be accessed immediately through an exposed HTTP port.

3.8 Concurrency

The concurrency best practice explains that cloud-native applications should scale out using the process model. What does this mean? Let's imagine, rather than making a single significant process larger, we can create multiple processes and then distribute the service's load or application among different processes.

Vertical scaling (scale up) refers to increasing the hardware infrastructure (CPU, RAM). Horizontal scaling (scale out) refers to adding more instances of the application. When you need to scale, launch more microservice instances and scale out and not up. Figure 2.10 shows the difference between both types of scaling.

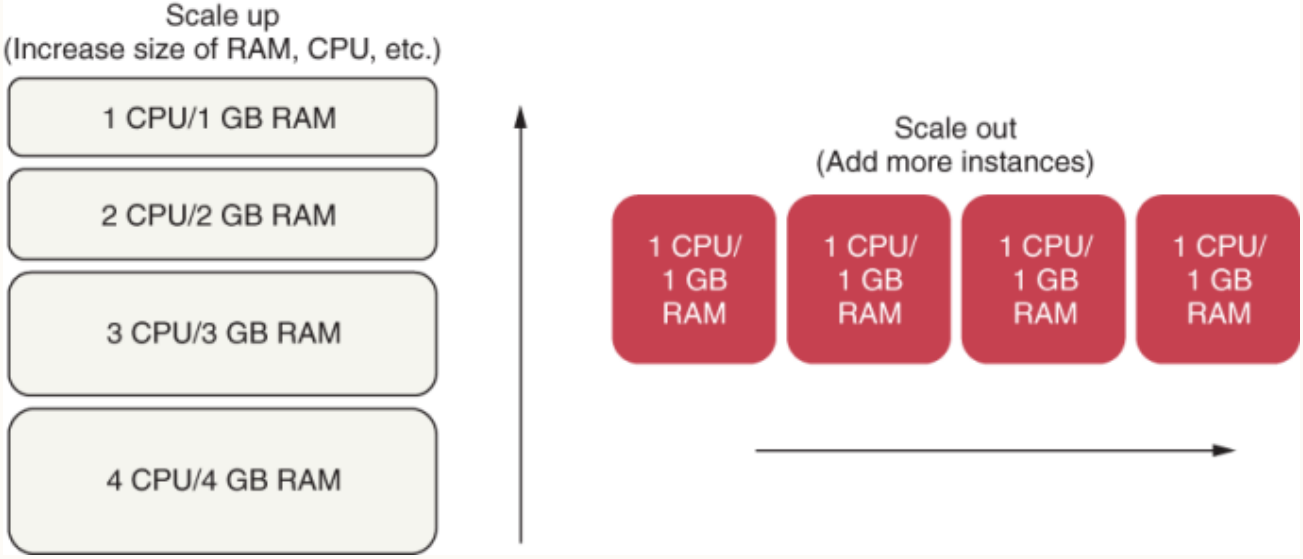


Figure 2.10 Differences between scaling up and scaling out

3.9 Disposability

Microservices are disposable and can start and stop on demand in order to facilitate elastic scaling and to quickly deploy application code and configuration changes. Ideally, startup should last a few seconds from the moment the launch command executes until the process is ready to receive requests.

What we mean by disposable is that we can remove failing instances with new instances without affecting any other services. For example, if one of the instances of the microservice is failing because of a failure in the underlying hardware, we can shut down that instance without affecting other microservices and start another one somewhere else if needed.

3.10 Dev/prod parity

This best practice refers to having different environments (for example, development, staging, production) as analogous as possible. The environments should always contain similar versions of deployed code, as well as the infrastructure and services. This can be done with continuous deployment, which automates the deployment process as much as possible, allowing a microservice to be deployed between environments in short periods of time.

As soon as code is committed, it should be tested and then promoted as quickly as possible from development all the way to production. This guideline is essential if we want to avoid deployment errors. Having similar development and production environments allows us to control all the possible scenarios we might have while deploying and executing our application.

3.11 Logs

Logs are a stream of events. As these are written, logs should be managed by tools such as Logstash (**https://www.elastic.co/logstash**) or Fluentd (**https://www.fluentd.org/**), which collect the logs and write those to a central location. The microservice should never be concerned about the mechanisms of how this happens. It only needs to focus on writing the log entries into the standard output (stdout).

In chapter 11, we will demonstrate how to provide an autoconfiguration for sending these logs to the ELK Stack (Elasticsearch, Logstash, and Kibana). Figure 2.11 shows how logging works in a microservice architecture using this stack.

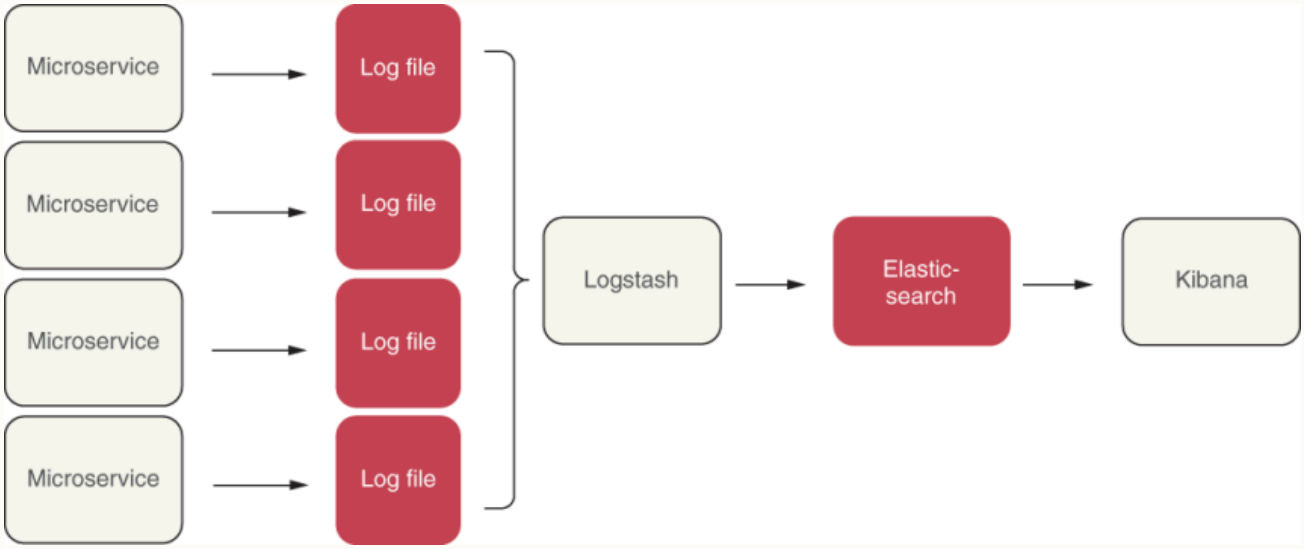


Figure 2.11 Managing the microservice logs with the ELK architecture

3.12 Admin processes

Developers will often have to do administrative tasks for their services (data migration or conversion, for example). These tasks should never be ad hoc and instead should be done via scripts that are managed and maintained through a source code repository. The scripts should be repeatable and non-changing (the script code isn't modified for each environment) across each environment they're run against. It's important to have defined the types of tasks we need to take into consideration while running our microservice, so that if we have multiple microservices with these scripts, we are able to execute all of the administrative tasks without having to do this manually.

NOTE If you're interested in reading more about Heroku's twelve-factor manifesto, visit the twelve-factor app website (**https://12factor.net/**).

In chapter 8, we will explain how to implement these features using the Spring Cloud API Gateway. Now that we have seen what the best practices are, we can continue with

the next section, where we'll start building our first microservice with Spring Boot and Spring Cloud.

2.4 Making sure our examples are relevant

We want to make sure this book provides examples that you can relate to as you go about your day-to-day job. To this end, we've structured the chapters in this book and the corresponding code examples around a software product of a fictitious company called Optima Growth.

Optima Growth is a software development company whose core product, Optima Stock (that we'll refer to as O-stock), provides an enterprise-grade asset management application. It furnishes coverage for all the critical elements: inventory, software delivery, license management, compliance, cost, and resource management. Its primary goal is to enable organizations to gain an accurate point-in-time picture of their software assets. The company is approximately 12 years old.

The company wants to rebuild their core product, O-stock. While much of the business logic for the application will remain in place, the application itself will be broken down from a monolithic architecture to a much smaller microservice design, whose pieces can be deployed independently to the cloud. The replatforming involved with O-stock can be a "make or break" moment for the company.

NOTE The examples in this book won't build the entire O-stock application. Instead, we'll build specific microservices from the problem domain at hand and then create the infrastructure that will support these services. We'll do this by using various Spring Cloud (and some non-Spring Cloud) technologies.

The ability to successfully adopt a cloud-based microservice architecture will impact all parts of the technical organization, including the architecture, engineering (development), and operations teams. Input will be needed from each group and, in the end, these groups are probably going to need reorganization as the team reevaluates their responsibilities in this new environment. Let's start our journey with Optima Growth as we begin the fundamental work of identifying and building several of the microservices used in O-stock. Then we'll build these services using Spring Boot.

NOTE We understand that the architecture of an asset management system is complex. Therefore, in this book, we will only use some of its basic concepts, focusing on creating a complete microservice architecture with a simple system as an example. Creating a complete software asset management application is beyond the scope of this book.

2.5 Building a microservice with Spring Boot and Java

In this section, we will build the skeleton of a microservice called licensing service for the Optima Growth company mentioned in the previous section. We'll create all of the microservices using Spring Boot.

Spring Boot, as previously mentioned, is an abstraction layer over the Spring libraries that allows us to quickly build Groovy- and Java-based web applications and microservices with significantly less ceremony and configuration than in a full-blown Spring application. For the licensing service example, we'll use Java as our core programming language and Apache Maven as our build tool. Over the next sections, we will

1. Create the basic skeleton of the microservice and a Maven script to build the application
2. Implement a Spring bootstrap class that will start the Spring container for the microservice and initiate the kick-off of any initialization work for the class

2.5.1 Setting up the environment

To start building our microservices, you should have the following components:

- Java 11 (**http://mng.bz/ZP4m**)
- Maven 3.5.4 or later (**https://maven.apache.org/download.cgi**)
- Spring Tools 4 (**https://spring.io/tools**) or you can download it within your selected Integrated Development Environment (IDE)
- IDEs such as
  - Eclipse (**https://www.eclipse.org/downloads/**)
  - IntelliJ IDEA (**https://www.jetbrains.com/idea/download/**)
  - NetBeans (**https://netbeans.org/features/index.html**)

NOTE All of the code listings from now on will be created using Spring Framework 5 and Spring Boot 2. It is important to understand that we're not going to explain all the features of Spring Boot, we're just going to highlight the essential ones in order to create the microservice. Another important fact is that we'll use Java 11 in this book in order to reach the largest audience possible.

2.5.2 Getting started with the skeleton project

To begin, you'll create a skeleton project for O-stock's licensing service using Spring Initializr. Spring Initializr (**https://start.spring.io/**) enables you to create a new Spring Boot project with the possibility of choosing dependencies from an extensive list. Also, it

allows you to change specific project configurations that you are about to create. Figures 2.12 and 2.13 show how the Spring Initializr page should look like for the licensing service.

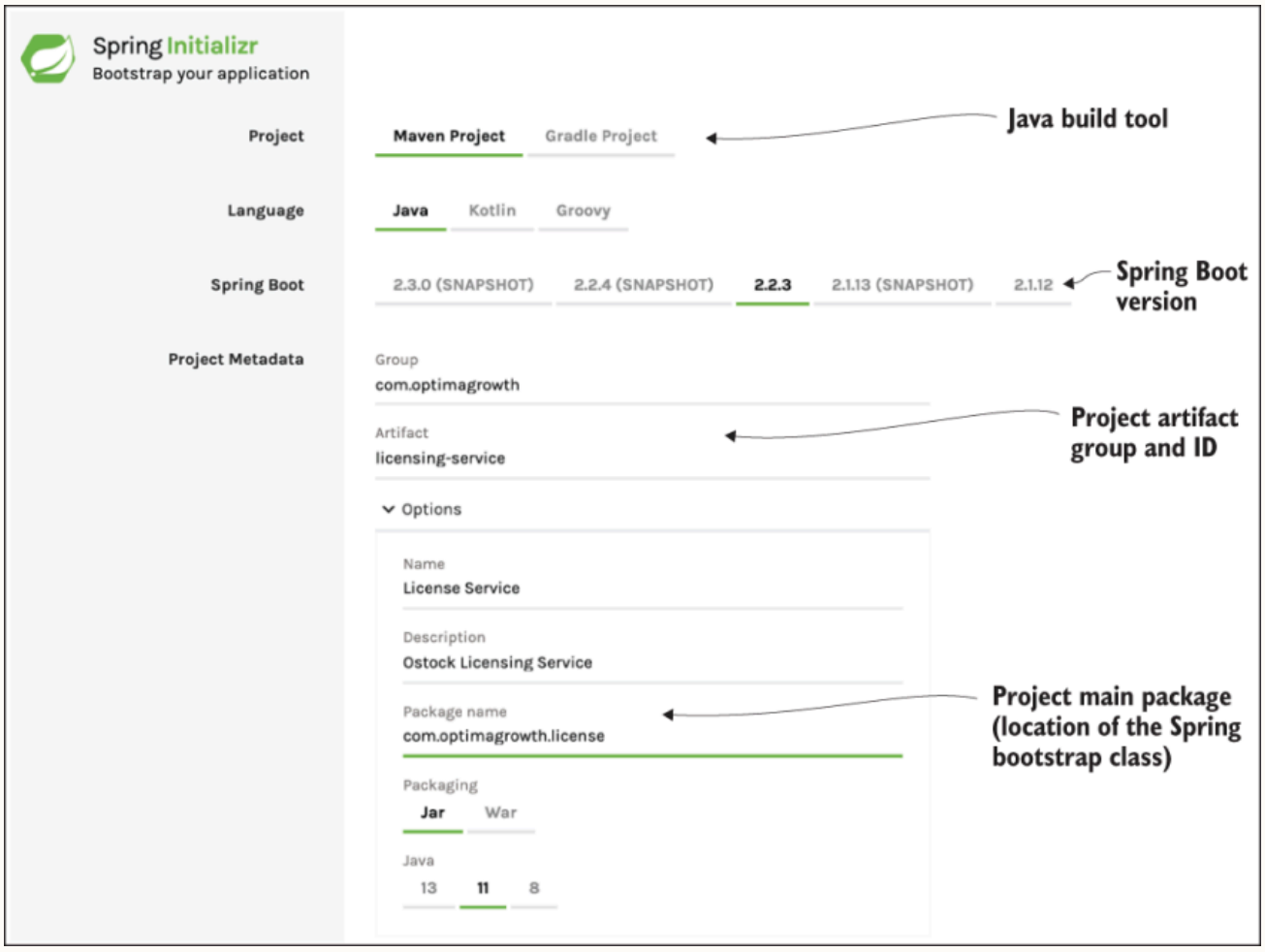


Figure 2.12 Spring Initializr dependencies for the licensing service

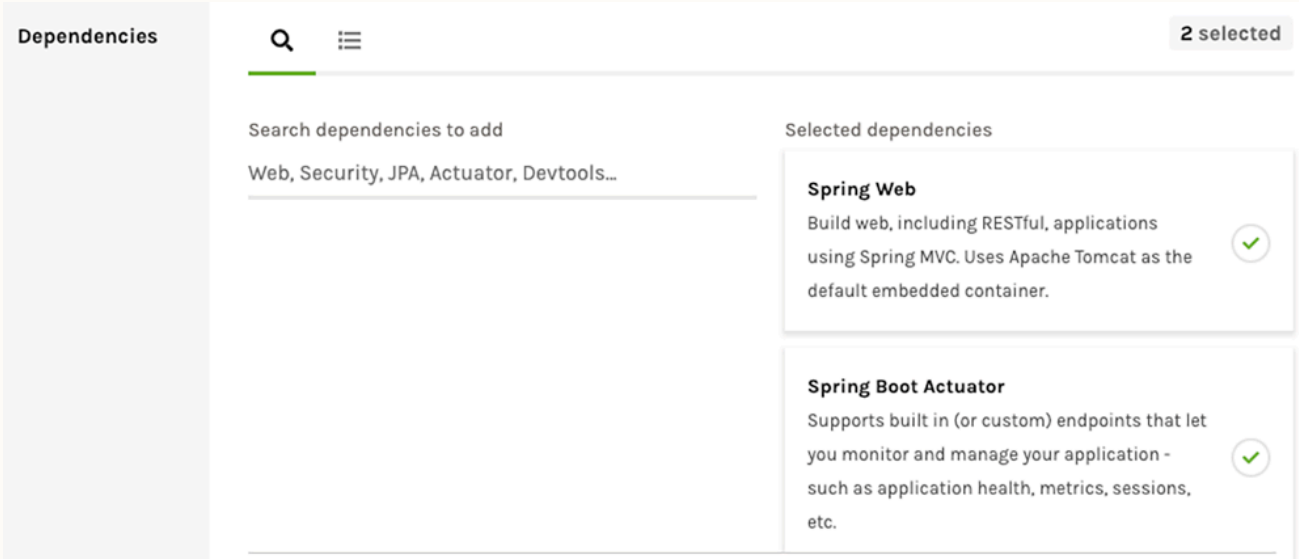


Figure 2.13 Spring Initializr configuration for the licensing service

NOTE You can also pull down the source code from the GitHub repository here: https://github.com/ihuaylupo/manning-smia/tree/master/chapter2.

Once you've created and imported the project as a Maven project into your preferred IDE, let's add the following packages:

```
com.optimagrowth.license.controller
com.optimagrowth.license.model
com.optimagrowth.license.service
```

Figure 2.14 shows the initial project structure for the licensing service in your IDE. Listing 2.2 then shows how your pom.xml file for our licensing service should look.

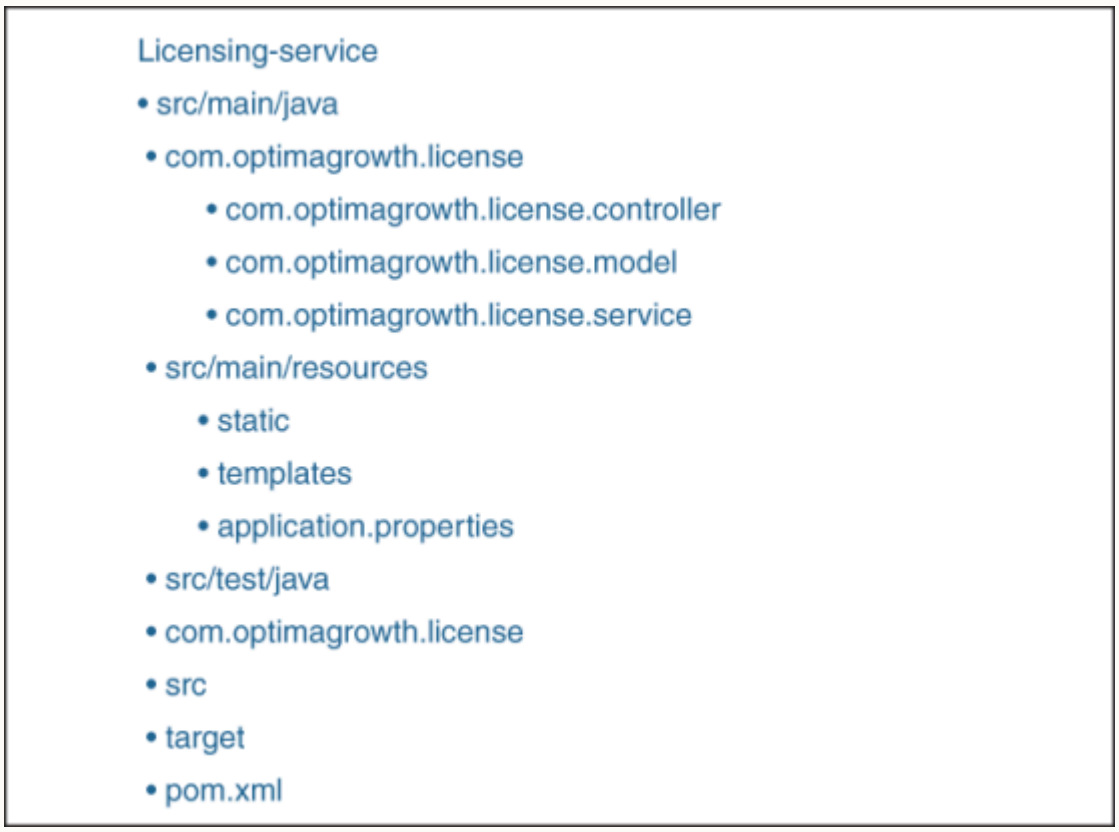


Figure 2.14 Licensing project structure for O-stock with the bootstrap class, application properties, tests, and pom.xml

NOTE An in-depth discussion of how to test our microservices is outside of the scope of this book. If you're interested in diving into more detail on how to create unit, integration, and platform tests, we highly recommend Alex Soto Bueno, Andy Gumbrecht, and Jason Porter's book, *Testing Java Microservices* (Manning, 2018).

Listing 2.2 Maven pom file for the licensing service

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
```

```
      https://maven.apache.org/xsd/maven-4.0.0.xsd">
<modelVersion>4.0.0</modelVersion>
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>                                    (1)
    spring-boot-starter-parent
  </artifactId>
  <version>2.2.3.RELEASE</version>
  <relativePath/> <!-- lookup parent from repository -->
</parent>
<groupId>com.optimagrowth</groupId>
<artifactId>licensing-service</artifactId>
<version>0.0.1-SNAPSHOT</version>
<name>License Service</name>
<description>Ostock Licensing Service</description>

<properties>
  <java.version>11</java.version>                 (2)
</properties>

<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>                                  (3)
      spring-boot-starter-actuator
    </artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>                                  (4)
      spring-boot-starter-web
    </artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
    <exclusions>
      <exclusion>
        <groupId>org.junit.vintage</groupId>
        <artifactId>junit-vintage-engine</artifactId>
      </exclusion>
    </exclusions>
  </dependency>
  <dependency>
    <groupId>org.projectlombok</groupId>
```

```
        <artifactId>lombok</artifactId>
        <scope>provided</scope>
      </dependency>
    </dependencies>
      <build>
      <plugins>
        <plugin>
          <groupId>org.springframework.boot</groupId>
          <artifactId>                                    ❺
            spring-boot-maven-plugin
          </artifactId>
        </plugin>
      </plugins>
    </build>
  </project>
```

❶ Tells Maven to include the Spring Boot starter kit dependencies

❷ By default, pom adds Java 6. To use Spring 5, we override it with Java 11.

❸ Tells Maven to include the Spring Actuator dependencies

❹ Tells Maven to include the Spring Boot Web dependencies

❺ Tells Maven to include Spring-specific Maven plugins for building and deploying Spring Boot applications.

NOTE A Spring Boot project doesn't need to set the individual Spring dependencies explicitly. These dependencies are automatically pulled from the Spring Boot core artifact defined in the pom file. Spring Boot v2.x builds use Spring framework 5.

We won't go through the entire file in detail, but we'll look at a few key areas. Spring Boot is broken into many individual projects. The philosophy is that you shouldn't have to "pull down the world" if you aren't going to use different pieces of Spring Boot in your application. This also allows the various Spring Boot projects to release new versions of code independently of one another.

To help simplify the life of the developers, the Spring Boot team gathered related dependent projects into various "starter" kits. In the listing, ❶ of the Maven pom file tells Maven that you need to pull down a specific version of the Spring Boot framework (in our case, 2.2.3). In ❷, you specify the version of Java you're going to use, and in ❸ and ❹, you identify that you're pulling down the Spring Actuator and Spring Boot Web

starter kits. Note that the Spring Actuator dependency is not required, but we will use several Actuator endpoints in the next chapters, so that's why we add it at this point. These two projects are the heart of almost any Spring Boot REST-based service. You'll find that as you build more functionality into your services, the list of these dependent projects becomes longer.

Spring also provides Maven plugins that simplify the build and deployment of Spring Boot applications. In ❺ of the pom file, you tell your Maven build script to install the latest Spring Boot Maven plugin. This plugin contains several add-on tasks (for example, `spring-boot:run`) that simplify your interaction between Maven and Spring Boot.

In order to check the Spring dependencies pulled by Spring Boot into our licensing service, we can use the Maven goal `dependency:tree`. Figure 2.15 shows the dependency tree for the licensing service.



Figure 2.15 Dependency tree for O-stock's licensing service. The dependency tree displays all of the dependencies declared and used in the service.

2.5.3 Booting your Spring Boot application: Writing the bootstrap class

Our goal in this section is to get a simple microservice up and running in Spring Boot and then iterate over it to deliver some functionality. To this end, you need to create two classes in your licensing service microservice:

- A Spring bootstrap class, which Spring Boot uses to start up and initialize the application
- A Spring controller class, which exposes the HTTP endpoints that can be invoked on the microservice

As you'll soon see, Spring Boot uses annotations to simplify setting up and configuring the service. This becomes evident as you look at the bootstrap class in the following code listing. You'll find this bootstrap class is in the LicenseServiceApplication.java file located in src/main/java/com/optimagrowth/license.

Listing 2.3 Introducing the `@SpringBootApplication` annotation

```
package com.optimagrowth.license;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication                          (1)
public class LicenseServiceApplication {

  public static void main(String[] args) {
    SpringApplication.run(                      (2)
      LicenseServiceApplication.class, args);
  }

}
```

(1) Tells the Spring Boot framework that this is the project's bootstrap class

(2) Starts the entire Spring Boot service

The first thing to note in this code is the use of the `@SpringBootApplication` annotation. Spring Boot uses this annotation to tell the Spring container that this class is the source of bean definitions. In a Spring Boot application, you can define Spring beans by

1. Annotating a Java class with a `@Component`, `@Service`, or `@Repository` annotation tag
2. Annotating a class with a `@Configuration` tag and then defining a factory method for each Spring bean you want to build with a `@Bean` tag

NOTE A Spring bean is an object that the Spring framework manages at run time with the Inversion of Control (IoC) container. These are created and added to a "repository of objects" so you can get them later.

Under the covers, the `@SpringBootApplication` annotation marks the application class in listing 2.3 as a configuration class. It then begins autoscanning all the classes on the Java classpath for other Spring beans.

The second thing to note in listing 2.3 is the `main()` method in the `LicenseServiceApplication` class. The `SpringApplication.run(LicenseServiceApplication .class, args)` call in the `main()` method starts the Spring container and returns a Spring `ApplicationContext` object. (We aren't doing anything with the `ApplicationContext`, so it isn't shown in the code.)

The easiest thing to remember about the `@SpringBootApplication` annotation and the corresponding `LicenseServiceApplication` class is that it's the bootstrap class for the entire microservice. Core initialization logic for the service should be placed in this class.

Now that we know how to create the skeleton and the bootstrap class for our microservice, let's continue with the next chapter. In the next chapter, we will explain some of the critical roles we must consider while we are building a microservice and how those roles are involved in the creation of the O-stock scenario. Also, we will explain some additional technologies to make our microservices more flexible and robust.

mmary

- Spring Cloud is a collection of open source technologies from companies like Netflix and HashiCorp. This technology is "wrapped" with Spring annotations to significantly simplify the setup and configuration of these services.
- Cloud-native applications are built with scalable components like containers, deployed as microservices, and managed on virtual infrastructures through DevOps processes with continuous delivery workflows.

- DevOps is the acronym for development (Dev) and operations (Ops). It refers to a software development methodology that focuses on communication, collaboration, and integration between software developers and IT operations. The primary goal is to automate the process of software delivery and infrastructure changes at lower costs.
- The twelve-factor application manifesto, framed by Heroku, provides best practices you should implement when building cloud-native microservices.
- The best practices of the twelve-factor application manifesto include codebase, dependencies, configuration, backing services, build/release runs, processes, port binding, concurrency, disposability, dev/prod parity, logs, and admin processes.
- The Spring Initializr allows you to create a new Spring Boot project while choosing dependencies from an extensive list.
- Spring Boot is the ideal framework for building microservices because it lets you build a REST-based JSON service with a few simple annotations.

Previous chapter
< 1 Welcome to the cloud, Spring

Next chapter
3 Building microservices with Spring Boot >